ADAPTIVE RAG & AGENTIC BFF

End-to-End System Architecture, Complete User Stories & Scenario Execution Guide

---

Document Type: Comprehensive Technical Specification & Agile User Stories
Architecture Stack: LangGraph 0.5+ | FastAPI | Rust Actix-Web (BFF) | Qdrant/FAISS Vector DB | OpenAI GPT-4o | Tavily Search | MongoDB Motor | Streamlit
Status: Production Ready & Fully Specified | **Coverage:** 100% of System Workflows & Edge Cases

1. Executive Summary & Core Mission

Adaptive RAG is an enterprise-grade, agentic Retrieval-Augmented Generation system designed to solve the fundamental flaws of traditional/naive RAG. In conventional RAG setups, every user input is blindly embedded and retrieved against a vector database—causing irrelevant retrieval on conversational chit-chat, complete failure on queries requiring real-time current events, and hallucinated answers when retrieved documents contain ambiguous or low-similarity content.

This project pioneers a hybrid **Multi-Agent StateGraph Architecture** combined with an enterprise **Rust Backend-For-Frontend (BFF)**. Incoming queries are dynamically evaluated by an intelligent routing classifier that selects the optimal execution path across three channels: (1) **Index Pipeline** (vector search with automated document relevance grading and query rewriting), (2) **General LLM Pipeline** (direct answering for common sense and conversational queries), or (3) **Live Web Search Pipeline** (Tavily external search for real-time facts). Furthermore, all queries are protected behind a high-performance Rust gateway enforcing Argon2 password hashing, JWT session security, and SQLite user isolation.

Feature / Capability	Naive RAG Systems	Adaptive RAG + Agentic BFF
Query Routing	Fixed: Always vector searches regardless of question	**Dynamic Classifier:** Routes to Vector Index, General LLM, or Tavily Web Search
Relevance Grading	None: Sends whatever top-K chunks are retrieved	**Automated Grader:** Evaluates context relevance; triggers query rewriting if low
Self-Correction Loop	Unsupported: Produces low-quality or hallucinated responses	**Query Rewriter:** Reformulates question into search-optimized terms and retries
Real-Time External Knowledge	Fails completely if data is not in local vector DB	**Tavily Integration:** Gathers current live web results on demand
Authentication & Security	Rarely integrated; plain API calls	**Rust Actix BFF:** Argon2 password hashing, SQLite DB, JWT tokens & API token gate
Conversation Memory	Stateless or single in-memory buffer	**Persistent MongoDB Motor:** Session-isolated asynchronous message histories

2. System Architecture & Component Topology

The solution is engineered as a decoupled, multi-tier distributed application comprising three primary subsystems:

- **Tier 1 — Frontend Client (Streamlit UI):** Multi-page web portal offering secure authentication (Login/Registration), multi-turn chat interface, document uploader (PDF/TXT), and session management.

- **Tier 2 — Enterprise Backend-For-Frontend (Rust Actix-Web BFF):** High-throughput security proxy enforcing double-layer authentication (API Token + JWT), user persistence in SQLite (`users.db`), password hashing via Argon2, and secure upstream proxying to the Python core.
- **Tier 3 — Agentic AI Engine (FastAPI & LangGraph):** StateGraph workflow orchestrator hosting ReAct agents, FAISS/Qdrant vector stores, OpenAI GPT-4o models, Tavily Web Search, and MongoDB chat history persistence.

3. Complete User Stories & Acceptance Criteria

Below is the complete catalog of Agile User Stories spanning all architectural epics, functional requirements, and execution scenarios.

EPIC 1: Enterprise Gateway, Security & Session Management (Agentic BFF)

US-1.1: User Registration & Strong Password Hashing

As an unauthenticated user,
I want to register a unique account with a username and password via the frontend,
So that my account credentials are created and stored using zero-knowledge cryptographic hashing.

Acceptance Criteria (Gherkin):

Given the client holds a valid session API token from /api/init,
When a POST request is sent to /api/create_user with username and plaintext password,
Then the Rust BFF must hash the password using **Argon2id** algorithm,
And insert the user record into SQLite (users.db) with a unique UUID v4,
And return HTTP 200 OK with success confirmation, or HTTP 400 if username already exists.

US-1.2: User Authentication & JWT Issuance

As a registered user,
I want to log in using my credentials,
So that I receive a signed JSON Web Token (JWT) authorizing my chat queries and isolating my conversation memory.

Acceptance Criteria (Gherkin):

Given an existing user in users.db,
When the user submits credentials to /api/login,
Then the system verifies the password against the stored Argon2 hash,
And if verified, generates a cryptographically signed JWT containing user claims and expiration,
And returns HTTP 200 with {"jwt": "<token>"},
And if invalid, returns HTTP 401 Unauthorized.

US-1.3: Authenticated Chat Gateway Proxying

As an authenticated user,
I want my chat questions routed through the Rust BFF /api/chat endpoint,
So that the system validates my JWT, forwards the query to the Python FastAPI backend with my token as session_id, and returns the sanitized answer.

Acceptance Criteria (Gherkin):

Given a request to /api/chat with an Authorization: Bearer <JWT> header,
When JwtMiddlewareFactory validates the token signature,
Then the BFF proxies a POST request to http://127.0.0.1:8000/rag/query,
And extracts the assistant response content, returning {"answer": "<content>"}.

EPIC 2: Document Ingestion & Knowledge Indexing Pipeline

US-2.1: Multi-Format Document Ingestion with Metadata Enrichment

As a knowledge worker,
I want to upload PDF or TXT documents along with a summary description in the UI sidebar,
So that the system enhances my description using an LLM, chunks the document, and indexes it into the vector database.

Acceptance Criteria (Gherkin):

Given a PDF or TXT file and a required X-Description header,
When POSTed to /rag/documents/upload,
Then the backend validates the file extension (rejecting non-PDF/TXT with HTTP 400),
And calls enhance_description_with_llm() to expand the domain description and saves it to description.txt,
And splits the document using RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150),
And embeds chunks with OpenAI Embeddings into FAISS/Qdrant, returning {"status": true}.

US-2.2: Cold-Start Vector Store Initialization

As a system operator,
I want the retriever tool to initialize safely even before any document has been uploaded,
So that early queries do not crash the server with null-pointer or empty collection errors.

Acceptance Criteria (Gherkin):

Given a freshly booted server with no uploaded documents,
When get_retriever() is invoked during startup or initial classification,
Then the system creates a placeholder FAISS vector store containing a dummy initialization document,
And builds a valid LangChain retriever tool that cleanly returns without raising an unhandled exception.

EPIC 3: Adaptive Query Routing & Orchestration Scenarios

US-3.1 [Scenario A]: In-Domain Document Query (Route: 'index', Grade: 'yes')

As a user asking questions about uploaded business documents,
I want the system to detect that my query pertains to the indexed documents and retrieve relevant chunks,
So that I receive an accurate, grounded answer synthesized directly from my uploaded source files.

Execution Workflow & Acceptance Criteria:

1. **Query Analysis:** query_classifier node queries vector store context and invokes LLM with RouteIdentifier schema.
2. **Routing Decision:** Context contains matching keywords; classifier outputs route = 'index'.
3. **Retriever Node:** LangGraph routes to retriever node which invokes ReAct agent and retrieves top chunks.
4. **Relevance Grading:** grade node checks chunk relevance against user question. Evaluator returns binary_score = 'yes'.
5. **Generation:** doc_tool routes directly to generate node.
6. **Output & Memory:** Final answer is generated, saved to MongoDB chat history, and returned to client.

US-3.2 [Scenario B]: In-Domain Query with Self-Correction (Route: 'index', Grade: 'no' -> 'rewrite')

As a user asking an ambiguous or informally phrased question about indexed documents,
I want the system to grade the retrieved context and rewrite my query if initial retrieval was poor,
So that the system re-searches the vector index with optimized keywords instead of hallucinating.

Execution Workflow & Acceptance Criteria:

1. **Initial Routing:** Query classifier detects the topic belongs to the uploaded corpus (route = 'index').
2. **Initial Retrieval:** retriever node fetches candidate chunks based on ambiguous prompt.
3. **Negative Grading:** grade node determines retrieved chunks do not sufficiently answer the question (binary_score = 'no').
4. **Query Rewriting:** doc_tool routes to rewrite node; LLM rewrites query into targeted technical search terms.
5. **Secondary Retrieval:** Graph loops back to retriever node with latest_query = <rewritten_query>.
6. **Re-Grading & Generation:** High-similarity chunks are retrieved, graded as 'yes', and sent to generate.

US-3.3 [Scenario C]: General Knowledge & Chit-Chat (Route: 'general')

As a user asking general knowledge, coding syntax, or conversational pleasantries,
I want the system to bypass the vector database and external search entirely,
So that I receive instant, cost-effective responses without vector pollution or latency.

Execution Workflow & Acceptance Criteria:

1. **Query Analysis:** query_classifier inspects context and detects no relevance to uploaded documents.
2. **Routing Decision:** Recognizes query as general world knowledge / conversational; sets route = 'general'.
3. **Direct Execution:** routing_tool transitions state to general_llm node.
4. **Completion:** OpenAI GPT-4o generates direct response; state transitions to END; MongoDB stores turn.

US-3.4 [Scenario D]: Real-Time External Web Search (Route: 'search')

As a user asking for real-time news, live events, or external data not in uploaded documents,
I want the system to execute a live web search via Tavily,
So that I receive up-to-date, factually accurate answers rather than stale LLM knowledge.

Execution Workflow & Acceptance Criteria:

1. **Query Analysis:** query_classifier finds no match in vector store and notes question requires real-time facts.
2. **Routing Decision:** Sets route = 'search'.
3. **Web Search Execution:** routing_tool routes to web_search node which invokes Tavily Search API.
4. **Synthesis:** Extracted web snippet contents are forwarded to generate node to synthesize a coherent summary.
5. **Completion:** Final response returned to user with real-time web grounding.

EPIC 4 & 5: Fact Verification, Memory & Resilience

• **US-4.1 (Faithfulness Fact-Checking):** For index-generated answers, verify_answer() inspects context vs final answer. If faithful = false, the system rejects the answer and re-triggers generation to eliminate hallucinations.
• **US-4.2 (Multi-Turn Asynchronous Memory):** All user prompts and AI responses are persisted in MongoDB Motor keyed by session_id. Upon subsequent turns, full message history is retrieved and passed into LangGraph to preserve conversational pronouns.
• **US-4.3 (Fault Tolerance & Graceful Degradation):** If MongoDB is unreachable, the system falls back to in-memory history. If unsupported files (.exe, .zip) are uploaded, the API responds with HTTP 400 without crashing worker threads.

4. Concrete End-to-End Scenario Walkthroughs with Real Examples

This section details real, concrete payloads, prompts, intermediate state objects, and responses for every execution path.

Scenario 1: Document Upload & Vector Indexing Ingestion

Context: The administrator uploads Employee_Handbook_2026.pdf detailing leave policies.

```
1. Client HTTP Request: POST /rag/documents/upload HTTP/1.1 Host: 127.0.0.1:8000 X-Description: Corporate Employee Handbook covering 2026 annual leave, sick days, and healthcare benefits. Content-Type: multipart/form-data; boundary=---Boundary123 [File Bytes: Employee_Handbook_2026.pdf (48 KB)] 2. Backend Processing: - LLM Description Enhancement: "This corporate document provides official HR guidelines, policy rules, vacation accrual schedules, and sick leave entitlement criteria for fiscal year 2026." -> Written to description.txt - Document Chunking: PyPDFLoader extracts 4 pages -> Split into 14 chunks (1000 chars, 150 overlap). - Vector Embedding: OpenAIEmbeddings converts 14 chunks into 1536-dimensional vectors. - Vector Store Storage: FAISS.from_documents() updates global index. - Dynamic Tool Registration: create_retriever_tool() updates retriever description. 3. Backend HTTP Response: HTTP/1.1 200 OK Content-Type: application/json {"status": true}
```

Scenario 2: In-Domain Query with High Relevance (Route: 'index', Grade: 'yes')

Context: Employee asks about vacation rollover rules defined in the uploaded handbook.

```
1. Client POST via Rust BFF /api/chat: {"query": "How many unused annual leave days can I carry over into next year?"} 2. LangGraph Execution Trace: [query_analysis node] Question: "How many unused annual leave days can I carry over into next year?" Retrieved context preview: "Section 4.2 Vacation Accrual: Up to 5 unused PTO days may roll over..." RouteIdentifier LLM Output: {"route": "index"} [retriever node] ReAct Agent Tool Call: retriever_customer_uploaded_documents("annual leave carry over policy") Tool Output: "Employees may carry over a maximum of 5 unused annual leave days into the next year. All rollover days must be utilized before March 31st." [grade node] GradingPrompt Evaluation: Context directly addresses the rollover limit and rules. Grade LLM Output: {"binary_score": "yes"} [generate node] Synthesizes clean final response from retrieved chunk. 3. Final Response JSON returned to Client: { "answer": "According to the Employee Handbook, you can carry over a maximum of 5 unused annual leave days into the next calendar year. Please note that all rolled-over days must be utilized prior to March 31st." }
```

Scenario 3: Self-Correction Loop (Route: 'index', Grade: 'no' -> Query Rewrite)

Context: User asks a slang/informal question: *'What happens if I get hurt outside work?'*

```
1. Initial State & Analysis: - Query: "What happens if I get hurt outside work?" - Classifier identifies HR policy topic -> route = "index" 2. Initial Retrieval & Failing Grade: - Retriever returns generic chunks on workplace accident reporting (OSHA standards). - Grade Node: question is about *non-work-related injury/short-term disability*, not OSHA. - Grade Output: {"binary_score": "no"} 3. Self-Correction Cycle: - Conditional Edge doc_tool routes to 'rewrite' node. - Rewrite Node: Input Query: "What happens if I get hurt outside work?" LLM Output: "short-term disability coverage and non-work-related illness medical leave policy" - Graph routes back to 'retriever' with optimized search query! - Second Retrieval: Fetches Section 8.3: "Non-Occupational Disability and Medical Leave". - Grade Node: {"binary_score": "yes"} -> routes to 'generate'. 4. Final Output to User: "If you experience a non-work-related injury, Section 8.3 entitles you to Short-Term Disability (STD) benefits covering up to 60% of base salary after a 7-day waiting period, provided medical documentation is submitted to HR."
```

Scenario 4: General Knowledge Query (Route: 'general')

```
1. User Query: "What is the difference between a Python tuple and list?" 2. Execution Path: - Vector DB context is completely irrelevant. - Query requires standard programming knowledge, not document-specific or live news. - Classifier Output: {"route": "general"} - Conditional Edge routes directly to 'general_llm' node (Vector search & Tavily bypassed). - OpenAI GPT-4o generates explanation (Lists are mutable; tuples are immutable). - Saved to MongoDB session history -> Returned to user in < 800ms.
```

Scenario 5: Real-Time Web Search Query (Route: 'search')

```
1. User Query: "What was the headline news about NASA's Artemis mission this week?" 2. Execution Path: - Vector DB context is irrelevant. - Query cannot be answered by static LLM knowledge (requires live 2026 news). - Classifier Output: {"route": "search"} - Routing tool sends state to 'web_search' node. - TavilySearchResults invokes live web crawler; retrieves 3 recent news articles. - Extracted text passed to 'generate' node to produce concise summary with sources. - Final grounded answer returned to user.
```

5. API & Technical Interface Contracts

The table below enumerates all public REST endpoints across both the Rust BFF and FastAPI services:

Service & Method	Endpoint Path	Headers / Auth	Description & Status Codes
Rust BFF POST	/api/init	None (Public)	Issues an initial session API token. **200 OK:** {"api_token": "..."}
Rust BFF POST	/api/create_user	X-API-TOKEN	Registers user with Argon2 password hashing. **200 OK:** User created | **400:** Exists
Rust BFF POST	/api/login	X-API-TOKEN	Authenticates credentials, returns signed JWT. **200 OK:** {"jwt": "..."} | **401:** Invalid
Rust BFF POST	/api/chat	Authorization: Bearer <JWT>	Proxies chat request to Python backend with session tracking. **200 OK:** {"answer": "..."}
Python Core POST	/rag/documents/upload	X-Description	Uploads & indexes PDF/TXT file into vector store. **200 OK:** {"status": true} | **400:** Bad type
Python Core POST	/rag/query	Content-Type: application/json	Direct RAG query interface with LangGraph orchestration. **200 OK:** {"result": {...}}

6. Configuration & Runtime Deployment

The system requires the following environment variables configured in .env:

- OPENAI_API_KEY: OpenAI API key for GPT-4o and Text-Embedding-3.
- TAVILY_API_KEY: Tavily API key for real-time web search.
- MONGODB_URL & MONGODB_DB_NAME: MongoDB connection string for asynchronous chat memory.
- QDRANT_URL & QDRANT_API_KEY: Vector database connection parameters (FAISS is used locally).

Execution Commands:

1. Start FastAPI Core: python -m uvicorn src.main:app --host 0.0.0.0 --port 8000 --reload
2. Start Auth & Gateway Service: python run_auth_service.py (Port 8080)
3. Start Streamlit UI: streamlit run streamlit_app/home.py --server.port 8501

SPECIFICATION SIGN-OFF & APPROVAL:
This specification represents the authoritative functional, behavioral, and architectural contract for the Adaptive RAG and Agentic BFF system. All defined scenarios are fully supported by the codebase implementation.